

RDTs(BIP-110), 비트코인 커뮤니티 거버넌스 논쟁의 본질은 무엇인가 ?

: 블록 체인(Block Chain) 정의, OP_RETURN 개념, 비트코인 블록 체인 데이터 입력 표현 분석, BIP-110 코드리뷰 까지



[Table Of Contents]

[Intro : Background , Motivation]

[핵심 질문 키워드 : 사이퍼펑크 운동(Cypherpunk Movement), 프로토콜 규칙 변경(Consensus Rule Change), 확장성과 BIP110 현상 (BIP-110 and Scalability)]

[Core Contents]

[Phase 0 : 'BIP110' 의 제한 금지, 목록, 동기 요약]

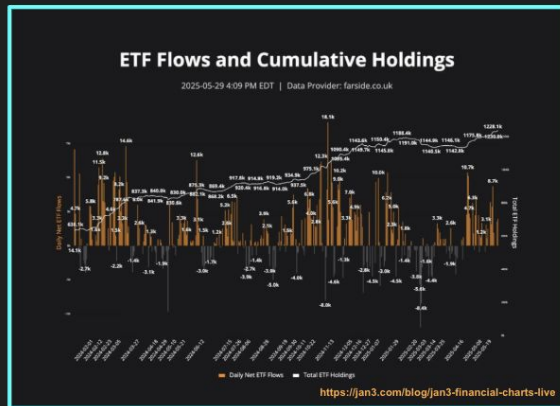
[Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?]

[Phase 2 : 왜 OP_RETURN 활용 데이터 삽입은 가장 중요하고 시급한 문제가 아닌가?]

[Phase 3 : BIP-110 구현 코드 비판적 읽기]

[Outro : Wrap it up]

[Intro : Background , Motivation]



[비트코인의 과거와 현재 그리고 미래 : 라이트닝 네트워크 중심으로 비트코인 톨아보기, 2024.1, orakle 3th conference]

["비트코인 블록체인에서 톨큰 발행 활발"...라이트닝 네트워크 확장, 2024.1, 디센터 <https://www.decenter.kr/article/13830202>]

세그윗 소프크포크 진행시 프로토콜 변경사항 추적 연구를 진행하며 결과적으로 라이트닝 네트워크가 사토시 단위의 소액 결제를 1초 안에 처리하고 기존 글로벌 결제망보다 현저히 낮은 수수료를 제공하며, 제도권에서 RGB·Taro와 같은 자산 발행 프로토콜 수용에 대해서 기술적·데이터적 근거를 바탕으로 비트코인 중심의 네이티브 금융이 결제를 넘어 자산 발행과 금융 서비스로 확장될 가능성을 제시했음. 이후 2025년에 제시된 2024년도 데이터 기준 ETF를 통한 전통 금융권의 자본 유입과 JAN3의 국가 채택 자료에서 확인되는 국가별 비트코인 보유·채굴·법정화폐 실험, 미국의 전략적 비트코인 준비자산 체제는 당시의 전망이 개인 결제망의 성장을 넘어 기관 금융과 국가 전략자산 체제로 본격화되고 있음을 보여줌.

Cypherpunk Is dead ... ?

[Intro : Reflection on “Cypherpunk Is dead ... ?”]

최근 “Cypherpunk is dead?”라는 밈과 BIP-110을 둘러싼 극명한 찬반을 바라보며 현재의 맥락에서 사이퍼펑크 정신이 무엇인지(혹시 하나의 고정된 교리나 동일한 정치적 입장을 뜻하는지) 다시 묻게 되었음. 개인의 자유와 프라이버시 기술 개발을 통해 초기 사이퍼펑크의 활동이 비트코인 으로 확장된 오늘날의 비트코인 생태계에서 과연 어떤 가치를 보호하거나 추구해야 하는지를 두고 서로 다른 결론에 도달해 있다고 보자면, 어떤 이들은 임의 데이터 제한을 비트코인의 화폐적 기능과 탈중앙성을 지키는 방어로 보고 다른 이들은 사용자가 선택할 수 있는 프로토콜의 중립성과 무허가성을 훼손하는 통제로 볼 수 있음. 그런데 이 분열을 추적하면서 느낀 것은 사이퍼펑크가 죽었다기보다는, 그 정신의 계승자가 누구이며 자유·중립성·책임 가운데 무엇을 우선해야 하는지를 둘러싼 논쟁 속에서 다시 시험받고 있다는 것 같다고 의구심을 갖게 됨.

[Intro : Reflection on “Cypherpunk Is dead ... ?” - Wrap It Up]

A. “Cypherpunk is dead?” → 사이퍼펑크 정신이 무엇인지? → 현재, 그 정신의 계승자가 누구이며 자유·중립성·책임 가운데 무엇을 우선해야 하는지를 둘러싼 논쟁 속에서 다시 시험받고 있다는 것은 아닌지 ?

Phase 0 : ‘BIP110’ 의 제한 동기, 금지 목록, 근거 요약

[a. 초록(abstract) | b. 명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

Phase 0 : 'BIP110' 의 제한 동기, 금지 목록, 근거 요약

[a.초록(abstract) | b.명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

Luke Dashjr (Luke-Jr) (2026.8.31 기준)

- 2011년부터 활동해 온 Bitcoin Core 개발자이자 전 BIP 편집자
- 채굴자가 블록 템플릿을 받아 구성할 수 있게 한 **getblocktemplate 표준(BIP 22-23)** 저자
- 초기 채굴 풀 **Eligius**를 운영했으며 2023년 이를 투명한 블록 템플릿과 비수탁형 보상을 지향하는 **OCEAN Mining** 재출범시킴.
- 2026년 8월 29일, BIP-110 중심 프로토콜 변경 이후 드러난 **채굴의 미래에 대한 비전 차이**를 이유로 OCEAN의 회장·CTO·이사직에서 사임하고 지분 전량을 매각
- 채굴 탈중앙화를 위한 새 프로젝트 **CONVOY**를 추진 예정 선언



2026.7 캡처 x 프로필 이미지



Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 동기 요약

[a. 초록(abstract) | b. 명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

비트코인을 화폐로서 개선하는 일(Bitcoin as money)에 우선순위를 다시 집중(refocus)하고 임의 데이터 지원을 표준화함으로써 발생한 왜곡된 인센티브를 바로잡기 위해 합의 수준(consensus level)에서 데이터 필드의 크기를 일시적으로 제한한다.

(Temporarily limit the size of data fields at the consensus level, in order to correct distorted incentives caused by standardizing support for arbitrary data, and to refocus priorities on improving Bitcoin as money.)

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 동기 요약

[a.초록(abstract) | b.명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

명세(Specification)

1년간 임시로 시행되는 배포 기간에 생성되는 블록은 다음의 추가 규칙에 따라 검사 :

1. 새 출력의 `scriptPubKey`가 34바이트를 초과하면 무효이다. 단, 첫 번째 연산 코드가 `OP_RETURN`인 경우에는 최대 83바이트까지 유효하다.
2. `OP_PUSHDATA*`의 페이로드와 `script-argument-witness-items` 이 256바이트를 초과하면 무효이다. 단, `BIP16 scriptSig`에 포함된 `redeemScript` 푸시는 예외이다.
3. 정의되지 않은 위트니스 버전 또는 `Tapleaf` 버전을 사용하는 지출은 무효이다. 즉, `Witness v0/BIP 141`, `Taproot/BIP 341` 또는 `P2A`가 아닌 버전을 사용하는 지출은 무효이다.
정의되지 않은 위트니스 버전으로 출력을 생성하는 것은 여전히 유효하다.
4. `Taproot annex`가 포함된 위트니스 스택은 무효이다.
5. 257바이트보다 큰 `Taproot` 컨트롤 블록은 무효이다. 이는 128개의 스크립트 리프를 가진 머클 트리에 해당한다.
6. 실행 여부와 관계없이 어느 위치에도 `OP_SUCCESS*` 연산 코드를 포함하는 `Tapscript`는 무효이다.
7. 실행 결과와 관계없이 `OP_IF` 또는 `OP_NOTIF` 명령을 실행하는 `Tapscript`는 무효이다.

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 동기 요약

[a.초록(abstract) | b.명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

기존 UTXO에 대한 소급 면제(UTXO grandfathering)

활성화 높이 이전에 생성된 UTXO를 지출하는 입력은 모든 새로운 규칙의 적용을 면제받는다. 이러한 소급 면제는 이번 소프트포크로 인해 기존 코인이 동결되거나 지출 불가능해지는 일이 없도록 보장한다. 활성화 전에 승인된 모든 UTXO는 전체 배포 기간 동안 이전과 정확히 같은 방식으로 언제든지 지출할 수 있다. 새로운 규칙은 활성화 높이 이후에 생성된 UTXO에만 적용된다. 소프트포크가 완료되면 모든 높이에서 생성된 UTXO가 다시 제한 없이 사용될 수 있다.

GetBlockTemplate

이 배포는 BIP 9에 정의된 GBT 배포 이름인 "reduced_data"를 사용한다. 의무 시그널링 기간에는 해당 배포의 비트가 "vbrequired"에 포함된다.

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 동기 요약

[a.초록(abstract) | b.명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

2022년에 처음 악용된 “인스크립션(inscription)” 해킹을 시작으로 비트코인 트랜잭션 안에 임의 데이터를 삽입하는 흐름이 나타났다. 이는 노드 운영자에게 상당하고 불필요한 부담을 만들어내고 개발의 초점과 인센티브를 비트코인의 근본 목적, 곧 건전하고 허가가 필요 없으며 국경 없는 화폐라는 목적에서 벗어나게 만들고 있다.

Starting with the "inscription" hack first exploited in 2022, a trend has emerged around embedding arbitrary data into Bitcoin transactions, creating significant unnecessary burdens on node operators and diverting development focus and incentives away from Bitcoin's fundamental purpose of being sound, permissionless, borderless money.

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 동기 요약

[a. 초록(abstract) | b. 명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

구체적으로 이 제안은 256바이트를 초과하는 연속적인 임의의 데이터를 삽입하는 모든 방법을 무효화한다. 또한 거의 전적으로 데이터 삽입에 악용되는 대형 scriptPubKey와 Tapleaf 형식을 무효화한다. 마지막으로 OP_RETURN 출력에 대해 오랫동안 정책상 유지되어 온 83바이트 제한을 합의 규칙 안에서 복원한다.

Specifically, this proposal invalidates all methods of embedding contiguous arbitrary data larger than 256 bytes; it also invalidates large scriptPubKey and Tapleaf formats that are abused almost exclusively for data embedding; and finally, it restores, in consensus, the long-established 83-byte policy limit on OP_RETURN outputs.

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 동기 요약

[a. 초록(abstract) | b. 명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

또한 이는 비트코인의 핵심 기능, 곧 검열 저항적인 결제 시스템이라는 기능을 강화한다. 데이터 저장은 결제와 불공정하게 경쟁하여 비트코인 결제를 불필요하게 비싸게 만들기 때문이다. 이는 제3자 결제 처리자에 대한 의존을 장려하고, 결과적으로 비트코인 결제를 더 쉽게 검열할 수 있게 만든다.

It also reinforces Bitcoin's core function as a censorship-resistant payment system, because data storage competes unfairly with payments, making Bitcoin payments unnecessarily costly. This encourages reliance on third-party payment processors, making Bitcoin payments easier to censor.

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 동기 요약

[a. 초록(abstract) | b. 명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

비트코인은 “한 가지 일을 하되, 그것을 잘해야 한다.” 데이터 저장을 거부함으로써 이 BIP는 비트코인 개발자들을 끝없는 범위 확장으로부터 해방시키고 정말 중요한 것, 즉 화폐로서 비트코인의 성공에 집중할 수 있게 한다.

Bitcoin should "do one thing, and do it well". By rejecting data storage, this BIP liberates Bitcoin developers from endless scope creep, enabling them to focus on what's really important: Bitcoin's success as money.

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 근거 요약

[a. 초록(abstract) | b. 명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

Changelog

- **2025.10** | 초안 — 데이터 제한 임시 소프트포크 제안
- **2025.11** | 구조 수정 — UTXO grandfathering 도입, 반응형 활성화 방식 제거
- **2025.11–2026.02** | 배포 확정 — 수정 BIP9, 55% 임계치·강제 시그널링·1년 후 **EXPIRED** 상태 도입
- **2026.06** | **Complete** — 명세 및 구현 준비 완료로 상태 변경
- **2026.08.09** | **Closed** — 체인 분리 후 채굴이 지속되지 못하면서 제안 종결

Phase 0 : 'BIP110' 의 제한 동기, 금지, 목록, 근거 요약

[a.초록(abstract) | b.명세(specification) | c. 동기(motivation) | d. 근거(rationale) | d. 기타(etc)]

2026.8.7 분리 체인과 후속 사건 요약

- BIP-110 노드는 높이 **961,632**부터 비트 4를 표시하지 않은 메인체인 블록을 거부하며 별도 체인으로 분리
- OCEAN을 통해 **961,632-961,633** 두 블록이 생성됐지만 추가 해시레이트가 따라오지 않아 체인이 거의 정지(2026.8.31 기준 8 블록 생성)
- 따라서 BIP-110은 **lock-in**이나 데이터 제한 규칙의 **activation**에 도달하지 못했다.
- 분기된 블록은 현재 비트코인 메인체인에서는 유효한 체인 기록으로 사용되지 않으며 분리 체인에 해시레이트가 높아져 최초 블록 부터 100블록 이상 채굴 되는 속도가 가속되지 않는 이상 coinbase 보상의 실질적 경제 가치는 없음.
- OCEAN은 설정 오류로 일부 Stratum 채굴자의 해시레이트가 약 18시간 동안 BIP-110 체인으로 전달되고 약 **0.3 BTC의 손실 보상**을 지급
- 8월 9일 BIP-110은 **Closed** 처리됐고 뒤이어 Luke는 BIP 편집자에서 제외되고 OCEAN 업무에서 휴직하고 8월 29일에는 회장·CTO·이사직과 지분을 모두 정리하고 새 채굴 탈중앙화 프로젝트 **CONVOY**를 예고.

[Phase 0 : 'BIP110' 의 제한 동기, 금지 목록, 근거 요약 - Wrap It Up]

B. 인스크립션을 악용된 해킹 사례로 정의함을 근거로 비트코인 개발의 근본 목적인 허가가 필요없고 국경없는 건전한 화폐라는 취지를 지키기 위해 비트코인을 화폐로서 개선하는 일에 우선순위를 다시 집중하고 임의 데이터 지원을 표준화하는 작업을 위해 7가지 제한 규칙 제안 (BIP110)

Phase 1 : ‘임의의 데이터’는 비트코인 블록 체인 어디에 들어갈 수 있나?

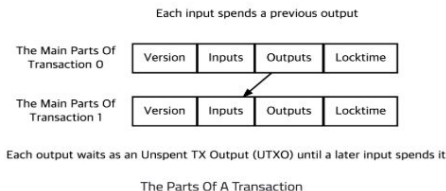
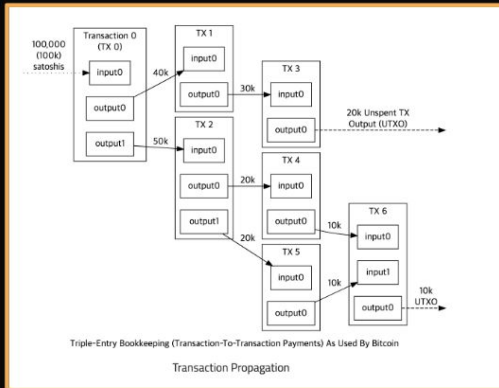
[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

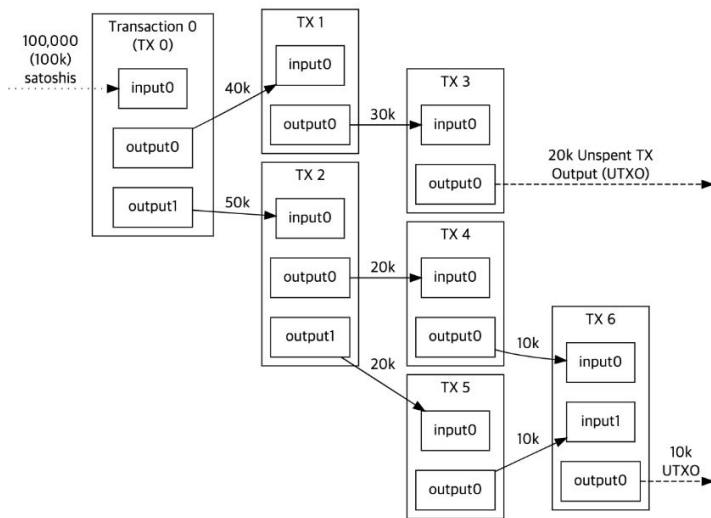
비트코인 블록 체인에 임의의 데이터가 들어가는 대표 경로는 출력의 **scriptPubKey**, null type인 **OP_RETURN**, 입력의 **scriptSig**, 세그윗 이후의 **witness**, witness 안에서 드러나는 **taproot script path**, 그리고 채굴자가 구성하는 **coinbase input script**이다. 다만 이들은 모두 동일한 성격이 아니며 어떤 것은 합의상 가능하지만 비표준이고 어떤 것은 표준 중계가 가능하며 어떤 것은 채굴자만 직접 넣을 수 있다.

```
Bitcoin block
├── transactions
│   ├── coinbase transaction
│   │   ├── coinbase input script
│   │   │   └── miner message / extra nonce / arbitrary data
│   └── normal transactions
│       ├── inputs
│       │   ├── scriptSig
│       │   │   └── legacy unlocking data / redeemScript / possible pushed data
│       │   └── witness
│       │       ├── signatures / pubkeys / scripts / stack items
│       │       └── taproot script path
│       │           └── tapscript / control block / inscription-style data
│       └── outputs
│           ├── scriptPubKey
│           │   ├── P2PKH / P2SH / P2WPKH / P2WSH / P2TR locking data
│           │   ├── custom scripts
│           │   └── OP_RETURN <data>
```



Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[Block Chain , Transaction , UTXO]



Triple-Entry Bookkeeping (Transaction-To-Transaction Payments) As Used By Bitcoin

Transaction Propagation

블록 체인(Block Chain)

블록 체인(Block Chain)은 비트코인의 공개 장부로서 거래들을 순서와 시간에 따라 기록한 것이다.

이 시스템은 이중 지불(double spending)을 방지하고 과거 거래 기록이 변경되는 것을 막기 위해 사용된다.

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

위치	구현 키워드	설명	현재 논쟁성
출력의 locking script	scriptPubKey	UTXO를 만들 때 붙는 잠금 스크립트. 임의의 바이트를 스크립트 형태로 넣을 수 있음	높음
OP_RETURN 출력	OP_RETURN	scriptPubKey 안에 OP_RETURN <data> 형태로 넣는 특수한 데이터 출력(<u>provably unspendable</u>).	높음
입력의 unlocking script	scriptSig	레거시 입력에서 서명, 공개키, redeemScript 등 넣는 공간 : 과거 임의의 데이터 삽입 <u>가능성</u> 컸지만 현재 표준 정책상 강하게 제한	중간
SegWit witness	witness	세그윗 입력의 서명, 공개키, 스크립트, 스택 데이터 공간 : Ordinals/Inscription의 핵심 무대	매우 높음
Taproot Script Path	taproot script path	사실상 witness 안에 드러나는 tapscript 경로 : 스크립트와 control block, witness stack에 데이터 들어감	매우 높음
coinbase input script	coinbase	블록의 첫 트랜잭션 입력 : 채굴자가 블록 높이, extra nonce, 메시지 등을 넣는 공간.	낮음

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

위치	구현 키워드	설명	현재 논쟁성
출력의 locking script	scriptPubKey	UTXO를 만들 때 붙는 잠금 스크립트. 임의의 바이트를 스크립트 형태로 넣을 수 있음	높음
OP_RETURN 출력	OP_RETURN	scriptPubKey 안에 OP_RETURN <data> 형태로 넣는 특수한 데이터 출력(<u>provably unspendable</u>).	높음
입력의 unlocking script	scriptSig	레거시 입력에서 서명, 공개키, redeemScript 등 넣는 공간 : 과거 임의의 데이터 삽입 <u>가능성</u> 컸지만 현재 표준 정책상 강하게 제한	중간
SegWit witness	witness	세그윗 입력의 서명, 공개키, 스크립트, 스택 데이터 공간 : Ordinals/Inscription의 핵심 무대	매우 높음
Taproot Script Path	taproot script path	사실상 witness 안에 드러나는 tapscript 경로 : 스크립트와 control block, witness stack에 데이터 들어감	매우 높음
coinbase input script	coinbase	블록의 첫 트랜잭션 입력 : 채굴자가 블록 높이, extra nonce, 메시지 등을 넣는 공간.	낮음

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

핵심은 OP_RETURN은 별도 영역이 아니라 scriptPubKey의 한 패턴이라는 점.

그리고 taproot script path 또한 별도 트랜잭션 필드라기보다 witness 안에서 공개되는 소비 경로.

즉,

$$\text{OP_RETURN} \subset \text{scriptPubKey}$$

$$\text{taproot script path} \subset \text{witness}$$

비트코인 raw transaction 구조에서 일반 입력은 scriptSig를 가지고, 출력은 scriptPubKey를 가지며,

scriptSig는 최대 10,000바이트, scriptPubKey도 최대 10,000바이트로 직렬화될 수 있음.

다만 이것은 “합의상 가능한 구조”에 가까운 이야기이고, 실제 네트워크 중계·채굴에는 표준성 정책이 별도로 작동.

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

OP_RETURN 예시, `bitcoin-cli getrawtransaction "0fe6a50d91f810016907c7bc58ed599b8badd3a92dcad5d3e4d50577c8867a4f" true | jq '.vout'`

```
[
  {
    "value": 0.00592900,
    "n": 0,
    "scriptPubKey": {
      "asm": "0 d751d5a56be7b6dd1d75a52ec39486dc395d7a75",
      "desc": "addr(tb1q6agatfttu7md68t455hv89yxmsu467n4lxz3t5)#x72hy2vd",
      "hex": "0014d751d5a56be7b6dd1d75a52ec39486dc395d7a75",
      "address": "tb1q6agatfttu7md68t455hv89yxmsu467n4lxz3t5",
      "type": "witness_v0_keyhash"
    }
  },
  {
    "value": 0E-8,
    "n": 1,
    "scriptPubKey": {
      "asm": "OP_RETURN ebb984ed8ab8ecbd94ec9db820ebb894eba19d20ecb2b4ec9db820eab8b0ec88a0ec9d9820eb8c80eca491ed9994eba5bc20ec9984ec84b1ed9598eb8ba4202d205465616d20426974636f696e",
      "desc": "raw(6a4c4debb984ed8ab8ecbd94ec9db820ebb894eba19d20ecb2b4ec9db820eab8b0ec88a0ec9d9820eb8c80eca491ed9994eba5bc20ec9984ec84b1ed9598eb8ba4202d205465616d20426974636f696e)#64jz2cn0",
      "hex": "6a4c4debb984ed8ab8ecbd94ec9db820ebb894eba19d20ecb2b4ec9db820eab8b0ec88a0ec9d9820eb8c80eca491ed9994eba5bc20ec9984ec84b1ed9598eb8ba4202d205465616d20426974636f696e",
      "type": "nulldata"
    }
  }
]
```


Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

OP_RETURN 중심 임의 데이터를 Provably Unspendable Output 속성화 하기 위한 Null Data 정의

Null Data 정의요약 :

Null Data 트랜잭션은 OP_RETURN을 이용해 임의 데이터를 사용할 수 없는 출력(provably unspendable output)에 기록하는 방식.

해당 출력은 소비될 수 없으므로 UTXO 세트(chainstate)에 추가되지 않으며 블록체인에는 영구 기록.

이유는 OP_RETURN으로 시작하는 스크립트가 provably unspendable로 판정되기 때문.

```
[ code : if (coin.out.scriptPubKey.IsUnspendable()) return; ]
```

합의 규칙상 scriptPubKey는 최대 10,000바이트까지 가능하지만 Bitcoin Core의 기본 중계·채굴 정책은 더 엄격하며 Core 0.9~0.10: 데이터 최대 40바이트, 단일 데이터 푸시, Core 0.11: 최대 80바이트, Core 0.12 이후: 스크립트 전체 최대 83바이트, 여러 데이터 푸시 허용, 기본적으로 OP_RETURN 출력은 하나이며 0사토시를 지급해야. -datacarriersize 옵션으로 중계·채굴할 Null Data 출력의 최대 크기를 설정할 수 있음.

즉, Null Data 트랜잭션의 OP_RETURN 출력도 트랜잭션과 블록에는 기록. 다만 명백히 소비 불가능한 출력으로 판정되므로

Bitcoin Core의 UTXO 세트에는 추가되지 않는다. 합의상 허용 범위와 노드의 기본 중계·채굴 정책은 서로 다름.

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

The first bitcoin transaction

Legacy bitcoin transactions have four main components: the version, inputs, outputs, and locktime. To illustrate each of these fields and what they do, we'll go through an example using the first ever bitcoin transaction. On January 11, 2009 at 7:30 PM PST, Satoshi Nakamoto transferred 10 BTC to Hal Finney.

This transaction spent a UTXO that was mined directly by Satoshi, where the block reward was 50 BTC. The transaction had two outputs, one to Hal Finney for 10 BTC, and a change output for 40 BTC.

Here is the raw serialized transaction in hex, as you'd see it in the blockchain:

```
0100000001c997a5e56e104102fa209c6a852dd90660a20b2d9c352423edce25857fcd3704000000004847304402204e45e16932b8
af514961a1d3a1a25fdf3f4f7732e9d624c6c61548ab5fb8cd410220181522ec8eca07de4860a4acd12909d831cc56cbbac46220822
21a8768d1d0901ffffffff0200ca9a3b00000000434104ae1a62fe09c5f51b13905f07f06b99a2f7159b2225f374cd378d71302fa28414e7
aab37397f554a7df5f142c21c1b7303b8a0626f1bade5c72a704f7e6cd84cac00286bee0000000043410411db93e1dcdb8a016b498
40f8c53bc1eb68a382e97b1482ecad7b148a6909a5cb2e0eaddfb84ccf9744464f82e160bfa9b8b64f9d4c03f999b8643f656b412a3ac
00000000
```

ChaincodeLabs tx-tutorial material

coinbase는 조금 다름. 일반 트랜잭션이 아니라 블록 안의 첫 트랜잭션 입력이고 이전 출력이 없음. **coinbase script**는 최대 100바이트이며

BIP34 이후에는 블록 높이가 먼저 들어가야 하고 남은 공간에 채굴자 메시지나 **extra nonce** 같은 임의 데이터가 들어갈 수 있음.

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | **c.witness** | d.coinbase | e.taproot script path | f.OP_RETURN]

witness는 세그윗 이후 생긴 입력별 증거 데이터 영역.

BIP141에 따르면 각 txin마다 witness field가 붙고, 그 안은 witness stack item들의 직렬화.

중요한 점은 witness data 자체는 script가 아니라 스택 아이템들의 직렬화라는 점.

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

Taproot script path에서는 이 witness 영역 안에 실행될 tapscript, control block, 그리고 스택 데이터가 드러남.

Ordinals/inscriptions는 대체로 이 구조를 이용해 실행상으로는 조건문 등으로 큰 데이터를 직접 의미 있는 지불 조건과 분리하면서도

witness에 데이터가 남게 만드는 방식입니다. BIP341은 Taproot를 SegWit v1 출력 타입으로 정의하고

BIP342는 Taproot scripts, 즉 tapscript 검증 규칙을 정의.

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[a.scriptPubKey | b.scriptSig | c.witness | d.coinbase | e.taproot script path | f.OP_RETURN]

Bitcoin block

transactions

coinbase tra

coinbase

mine

normal trans

inputs

scri

scri

witness

signatures / pubkeys / scripts / stack items

taproot script path

tapscript / control block / inscription-style data

outputs

scriptPubKey

P2PKH / P2SH / P2WPKH / P2WSH / P2TR locking data

custom scripts

OP_RETURN <data>

Taproot key path spending signature validation

A Taproot signature is a 64-byte Schnorr signature, as defined in [BIP340](#), with the sighash byte appended in the usual Bitcoin fashion. This sighash byte is optional. If omitted, the resulting signatures are 64 bytes, and a SIGHASH_DEFAULT mode is implied.

To validate a signature sig with public key q :

- If the sig is 64 bytes long, return $Verify(q, hash_{TapSighash}(0x00 || SigMsg(0x00, 0)), sig)^{20}$, where $Verify$ is defined in [BIP340](#).
- If the sig is 65 bytes long, return $sig[64] \neq 0x00^{21}$ and $Verify(q, hash_{TapSighash}(0x00 || SigMsg(sig[64], 0)), sig[0:64])$.
- Otherwise, fail²².

Design

A number of related ideas for improving Bitcoin's scripting capabilities have been previously proposed: Schnorr signatures ([BIP340](#)), Merkle branches ("MAST"; [BIP114](#), [BIP117](#)), new sighash modes ([BIP118](#)), new opcodes like [CHECKSIGFROMSTACK](#), [Taperoot](#), [Graftroot](#), [Groot](#), and [cross-input aggregation](#).

Combining all these ideas in a single proposal would be an extensive change, be hard to review, and likely miss new discoveries that otherwise could have been made along the way. Not all are equally mature as well. For example, cross-input aggregation *interacts* in complex ways with upgrade mechanisms, and solutions to that are still *in flux*. On the other hand, separating them all into independent upgrades would reduce the efficiency and privacy gains to be had, and wallet and service providers may not be inclined to go through many incremental updates. Therefore, we're faced with a tradeoff between functionality and scope creep. In this design we strike a balance by focusing on the structural script improvements offered by Taproot and Merkle branches, as well as changes necessary to make them usable and efficient. For things like sighashes and opcodes we include fixes for known problems, but exclude new features that can be added independently with no downsides.

Specification

This section specifies the Taproot consensus rules. Validity is defined by exclusion: a block or transaction is valid if no condition exists that marks it failed.

The notation below follows that of [BIP340](#). This includes the $hash_{SHA256}(x)$ notation to refer to $SHA256(SHA256(tag) || SHA256(x))$. To the best of the authors' knowledge, no existing use of $SHA256$ in Bitcoin feeds it a message that starts with two single $SHA256$ outputs, making collisions between $hash_{SHA256}$ with other hashes extremely unlikely.

Script validation rules

A Taproot output is a native SegWit output (see [BIP141](#)) with version number 1, and a 32-byte witness program. The following rules only apply when such an output is being spent. Any other outputs, including version 1 outputs with lengths other than 32 bytes, or P2SH-wrapped version 1 outputs³, remain unenumerated.

Signature validation rules

We first define a reusable common signature message calculation function, followed by the actual signature validation as it's used in key path spending.

Common signature message

The function $SigMsg(hash_type, ext_flag)$ computes the common portion of the message being signed as a byte array. It is implicitly also a function of the spending transaction and the outputs it spends, but these are not listed to keep notation simple.

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

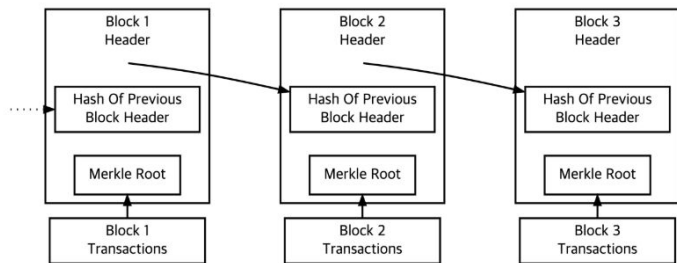
[Block Chain , Transaction, UTXO]

구현상 `CBlock`은 `CBlockHeader`를 상속하고, 네트워크/디스크에 저장되는 트랜잭션 목록으로 `std::vector<CTransactionRef> vtx`를 가짐.

“첫 번째 트랜잭션은 블록 생성자가 소유하는 새 코인을 만드는 특수 트랜잭션”이라고 주석에 설명되어 있음.

블록 체인(Block Chain): 블록 체인(Block Chain)은 비트코인의

공개 장부로서 거래들을 순서와 시간에 따라 기록한 것이다. 이 시스템은 이중 지불(double spending)을 방지하고 과거 거래 기록이 변경되는 것을 막기 위해 사용.



Simplified Bitcoin Block Chain

Block Chain Overview

Bitcoin Whitepaper's 9. Combining and Splitting Value

Although it would be possible to handle coins individually, it would be unwieldy to make a separate transaction for every cent in a transfer.

```
73 class CBlock : public CBlockHeader
74 {
75 public:
76     // network and disk
77     std::vector<CTransactionRef> vtx;
78
79     // Memory-only flags for caching expensive checks
80     mutable bool fChecked; // CheckBlock()
81     mutable bool m_checked_witness_commitment{false}; // CheckWitnessCommitment()
82     mutable bool m_checked_merkle_root{false}; // CheckMerkleRoot()
83
84     CBlock()
85     {
86         SetNull();
87     }
88
89     CBlock(const CBlockHeader &header)
90     {
91         SetNull();
92         *(static_cast<CBlockHeader*>(this)) = header;
93     }
94
95     SERIALIZE_METHODS(CBlock, obj)
96     {
97         READWRITE(AsBase<CBlockHeader>(obj), obj.vtx);
98     }
99
100     void SetNull()
101     {
102         CBlockHeader::SetNull();
103         vtx.clear();
104         fChecked = false;
105         m_checked_witness_commitment = false;
106         m_checked_merkle_root = false;
107     }
108
109     std::string ToString() const;
110 };
```

Phase 1 : 임의의 데이터는 비트코인 블록 체인 어디에 들어갈 수 있나?

[Block Chain , Transaction, UTXO]

구현상 `CBlock`은 `CBlockHeader`를 상속하고, 네트워크/디스크에 저장되는 트랜잭션 목록으로 `std::vector<CTransactionRef> vtx`를 가짐.

“첫 번째 트랜잭션은 블록 생성자가 소유하는 새 코인을 만드는 특수 트랜잭션”이라고 주석에 설명되어 있음.

개념	Bitcoin Core 타입 / 필드	파일 위치	의미
Bitcoin block	<code>class CBlock</code>	<code>src/primitives/block.h</code>	블록은 헤더를 상속하고, 트랜잭션 벡터 <code>vtx</code> 를 가진다.
Transactions	<code>std::vector<CTransactionRef> vtx</code>	<code>src/primitives/block.h</code>	블록 안의 트랜잭션 목록 첫 번째가 coinbase transaction 이다.

[Phase 1 : ‘임의의 데이터’는 비트코인 블록 체인 어디에 들어갈 수 있나? - Wrap It Up]

C. OP_RETURN을 활용한 데이터 삽입 방식은 Null Data 표준으로 증명가능하게 출력이 불가능한 형식을 정책적으로 구현하고 있으며 실제로 블록 체인에 데이터 입력이 가능한 표면이 Segwit Witness, Taproot Script Path 같은 공간이 총 5개 존재.

Phase 2 : 왜 OP_RETURN 활용 데이터 삽입은 해결해야할 시급한 문제가 아니었는가?

OP_RETURN의 'provably unspendable' + 'null data' 정의 정책 vs inscription류 witness/tapscripT 구조를 이용해서 데이터를 넣기

(단, 초기 오픈리턴 정책 규칙 도입 중심으로 29버전이하버전과 30이상 버전에서 나타난 차이 설명, -datacarrier, -datacarriersize 옵션 중심)

Phase 2 : 왜 OP_RETURN 활용 데이터 삽입은 해결해야할 시급한 문제가 아니었는가 ?

[OP_RETURN의 'provably unspendable' + 'null data' 정의 정책 우회 데이터 삽입 기법 출현, 실제 인스크립션 예시]

OP_RETURN과 인스크립션은 무엇이 다른가?

OP_RETURN은 임의 데이터를 넣기 위해 명시적으로 구분된 Null Data 출력이다. 해당 출력은 소비할 수 없으므로 UTXO 세트에 추가되지 않으며, Bitcoin Core는 `-datacarrier`와 `-datacarriersize` 같은 정책 옵션으로 중계·채굴 범위를 제한해 왔다. 반면 인스크립션은 Taproot 입력의 witness와 tapscript 안에 `OP_FALSE OP_IF ... OP_ENDIF` 구조로 데이터를 넣는다. 이 데이터 역시 UTXO 세트에는 저장되지 않지만 블록에는 기록되며, Bitcoin Core는 이를 '인스크립션'이 아니라 유효한 witness script로만 처리한다. 이미지·텍스트라는 의미는 `ord`와 같은 외부 인덱서가 부여한다. 따라서 두 방식의 핵심 차이는 **UTXO 저장 여부가 아니라, 데이터 삽입 경로가 명시적으로 구분되고 정책적으로 제한되는가, 아니면 일반적인 스크립트 경로가 외부 프로토콜에 의해 데이터 저장 용도로 해석되는가에 있다.**

Phase 2 : 왜 OP_RETURN 활용 데이터 삽입은 해결해야할 시급한 문제가 아니었는가 ?

[OP_RETURN의 'provably unspendable' + 'null data' 정의 정책 우회 데이터 삽입 기법 출현, 실제 인스크립션 예시]

inscription #8,719,943 해당 트랜잭션에서 일어나는 일

- 인스크립션 본문은 출력이 아니라 **Taproot 입력의 witness/tapscript**에서 공개.
- **OP_FALSE OP_IF** 분기 내부는 실행되지 않지만 그 안의 데이터는 **블록에 기록된다**.
- 데이터 자체는 UTXO 세트에 들어가지 않으며, **Bitcoin Core는 이를 이미지가 아닌 유효한 스크립트 데이터로 처리한다**.
- **ord** 인덱서가 **ord** 표식과 content-type을 읽어 이미지라는 의미를 부여한다.

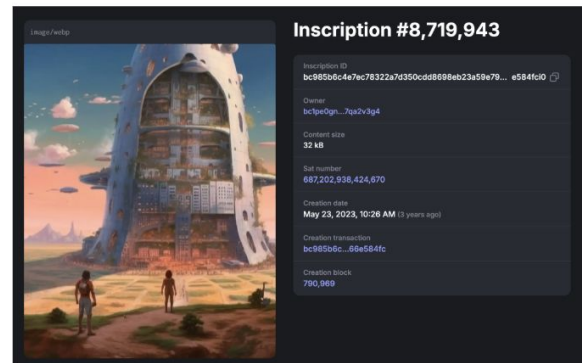
- 인스크립션 예시 오디 스캔검색 : <https://ordiscan.com/tx/bc985b6c4e7ec78322a7d350cdd8698eb23a59e7901936d143aaa28766e584fc>

- 멤풀. 스페이스내 검색 : <https://mempool.space/tx/bc985b6c4e7ec78322a7d350cdd8698eb23a59e7901936d143aaa28766e584fc?showDetails=true>

- hex 데이터 : <https://blockstream.info/api/tx/bc985b6c4e7ec78322a7d350cdd8698eb23a59e7901936d143aaa28766e584fc/hex>

- 루크에 의해 버그로 규정된 패턴 (CVE-2023-50428 , 하지만 모든 코어개발자들이 동의한것 아님)

02000000 Version | 0001 SegWit marker/flag | Witness | Signature | Control Block | Tapscript |
OP_FALSE OP_IF | "ord" | image/webp | <31,962-byte payload> | OP_ENDIF | 00000000 Locktime



실제 인스크립션 사례 이미지

Phase 2 : 왜 OP_RETURN 활용 데이터 삽입은 해결해야할 시급한 문제가 아니었는가 ?

[임의 데이터 허용 범위 논쟁을 합의 규칙의 문제로 확장한 BIP-110의 제안과 구현]

임의 데이터 제한의 합의 규칙화

Bitcoin Core v30의 정책 변경은 단순한 OP_RETURN 크기 조정을 넘어 **비트코인에서 임의 데이터의 허용 범위를 누가 어떤 계층에서 결정할 것인가**라는 논쟁으로 이어졌고, v29 이하의 83바이트 제한은 각 노드가 선택할 수 있는 중계·채굴 정책이었지만 BIP-110은 OP_RETURN 출력은 83바이트로 제한하고 일반 scriptPubKey, 데이터 푸시, witness item 등 다른 데이터 삽입 경로에도 제한을 적용하려 제안함.

즉, v30의 완화된 기본 정책을 이전 상태로 되돌리는 데 그치지 않고 인스크립션처럼 다른 스크립트 경로를 이용한 데이터 삽입까지 **합의 규칙 차원에서 제한하려는 제안**. 여기서 논쟁의 중심은 데이터 크기 자체보다 노드 운영자가 선택하던 정책의 영역을 모든 노드가 따라야 하는 합의 규칙으로 옮기는 것이 정당한가에 있음.

[Phase 2 : 왜 OP_RETURN 활용 데이터 삽입은 해결해야할 시급한 문제가 아니었는가? - Wrap It Up]

D.OP_RETURN의 'provably unspendable' + 'null data' 정의 정책을 우회하여 witness/tapscript 구조를 이용한 inscription 우회 데이터 삽입 기법이 출현하며 임의 데이터 허용 범위 논쟁을 정책적 문제가 아닌 합의 규칙의 문제로 확장한 RDTS(BIP-110 제안) 등장

Phase 3 : BIP-110 구현 코드 비판적 리뷰

Phase 3 : BIP-110 구현 코드 비판적 읽기

[Code Review]

본 리뷰는 BIP-110의 제안 배경과 구현 내용을 일반적으로 조사·요약하는 데 목적을 두지 않으며, Peter Todd가 제기한 poison block 및 검증 비용 증가 가능성 의혹에 대한 성립 여부와 증명 가능성에 대한 탐색을 출발점으로, grandfathering과 flags_per_input 도입이 full-script execution cache의 동작과 스크립트 검증 경로에 어떤 변화를 만드는지 추적한다. 이를 통해 캐시 재사용 제한, 스크립트 반복 실행, 실질적 비용 증폭으로 이어지는 논리가 코드상 성립하는지를 단계별로 검증하고, 근거가 확인되지 않는 주장은 명시적으로 수정하거나 철회하는 것을 목적으로 한다.

Phase 3 : BIP-110 구현 코드 비판적 읽기

【 왜 grandfathering이 필요한가 ? 】

문제의 출발점: 왜 grandfathering이 필요한가 ?

BIP-110의 제한 규칙을 활성화 이전에 생성된 UTXO에까지 소급 적용하면, 과거에는 유효했던 출력을 이후에 소비할 수 없게 될 수 있다. 이를 방지하기 위해 구현은 각 입력이 참조하는 UTXO의 생성 시점에 따라 적용 규칙을 달리하는 grandfathering을 도입한다.

그 결과 검증 규칙은 더 이상 트랜잭션 전체에 동일하게 적용되는 단일 플래그 집합만으로 표현되지 않는다. 동일 트랜잭션 안에서도 입력마다 서로 다른 규칙이 적용될 수 있으므로 `flags_per_input`이 필요해진다.

Phase 3 : BIP-110 구현 코드 비판적 읽기

[컨센서스 변경 코드 분석]

코드에서 확정된 변화

grandfathering 필요



입력별로 적용 규칙이 달라질 수 있음



flags_per_input 도입



트랜잭션 전체에 단일한 script flags를 전제한 full-script execution cache 사용 조건 변화

여기까지는 구현과 조건문을 통해 확인할 수 있는 코드.

Phase 3 : BIP-110 구현 코드 비판적 읽기

[full - script cache , signature cache 와 동작구조]

두 가지 캐시 관련 구현에 관하여 : full - script cache , signature cache 와 동작구조 - CPU L1 ~ L3 RAM 계층까지 동작 파악 목표.

a. Tx의 메모리 진입 때 이미 검증한 트랜잭션이 나중에 블록에 포함되었을 때, 동일한 비싼 스크립트·서명 검증을 반복하지 않게 하는 것

Signature cache : `VerifySignature(signature, pubkey, sighash) == true` $\Leftrightarrow$ signature + public key + signature hash/message

CHECKSIG

└ signature cache lookup

└ hit → true 반환

└ miss → libsecp256k1 검증

└ 성공 결과 cache 저장

>> 이 캐시가 절약하는 핵심 비용은 유한체 연산, 타원곡선 점 연산, 스칼라 곱셈을 포함하는 ECDSA 또는 Schnorr 검증

Script-execution cache : `CheckInputScripts(transaction, spent_outputs, flags) == true`

script-execution cache hit

↓

트랜잭션의 전체 입력 검증 성공을 바로 재사용

>> 캐시가 적중하면 개별 입력에 대해 다시 scriptSig 실행 | scriptPubKey 실행 | witness program 판정 | tapscript 실행 | OP_CHECKSIG | signature cache 조회 | 실제 secp256k1 검증 까지 내려가지 않아도 된다

b. full-script 캐시는 개념적으로 다음 명제를 저장해야. “ 이 트랜잭션은 이 입력 UTXO들을 소비할 때 이 flags 조합 아래에서 모든 입력 검증을 통과 ”

c. 단순화한 캐시 키 : Hash(transaction identity, spent-output context, script verification flags, process-local nonce)

Phase 3 : BIP-110 구현 코드 비판적 읽기

[Core Function Call Path from ConnectBlock() to libsecp256k1]

Core Function Call Path from ConnectBlock() to libsecp256k1

ConnectBlock() → grandfathering flag 분기 → CheckInputScripts() → Script Interpreter → OP_CHECKSIG → ECDSA/Schnorr → libsecp256k1 ,

>> Peter Todd's Criticism

" full-script cache 기존 로직에서, grandfathering flag 분기 적용을 하게 되면 '특정 상황'에서 old outputs 때문에 cache가 깨질 수 있고 모든 암호학 연산을 새롭게 진행해야 하는 비용을 증가시키는 **poison block 현상이 일어날 가능성**을 봐야 한다"고 주장

>> Peter Todd 의 주장을 입증하면 포이즌 블록 발생 hypothesis 성립

즉, BIP-110 grandfathering 구현 로직이 full-script cache를 우회했을 때 실제로 어느 수준까지 계산이 다시 수행되는가? 를 확인하고 측정.

Phase 3 : BIP-110 구현 코드 비판적 읽기

[실행 레벨에서의 가설 설정]

구분	명문화된 주장	현재 근거	증거 수준	다음 검증
A	<code>flags_per_input</code> 이 사용되는 검증 경로에서는 기존 <code>full-script execution cache</code> 를 정상적으로 조회·재사용할 수 없는 조건이 발생한다.	Peter Todd 리뷰와 정적 코드 분석	코드분석으로 확정 가능	캐시 조회·삽입을 결정하는 정확한 함수와 조건문 확정
B	캐시를 재사용하지 못하는 트랜잭션은 검증 문맥이 반복될 때 해당 입력의 스크립트 실행을 다시 수행할 수 있다.	A와 검증 호출 경로에서 도출	실험, 계측	<code>CheckInputScripts</code> 스크립트 실행 함수 호출 횟수 동적 계측
C	공격자가 <code>grandfathered input</code> 과 고비용 스크립트를 조합할 수 있다면, 블록 검증·재검증 과정에서 일반적인 캐시 경로보다 실질적인 CPU 비용을 증폭시킬 수 있다.	A와 B를 결합해 도출	체인분리 환경 재현, 공격자 가설 설정 후 실행	체인분리 환경 재현, 공격자 가설 설정 후 실행하여 <code>CheckInputScripts</code> 스크립트 실행 함수 호출 횟수 및 CPU, L 계층 캐시, RAM 정밀 동적 계측

Phase 3 : BIP-110 구현 코드 비판적 읽기

가설 A — 캐시 사용 조건의 변화

BIP-110의 grandfathering은 입력별 검증 플래그를 필요로 하며, 이로 인해 기존 full-script execution cache를 사용할 수 없는 실행 경로가 생긴다. (캐시 전체가 사라지는 것은 아니고 특정 조건에서 full-script execution cache의 조회 또는 재사용이 제한된다. 이것이 signature cache까지 모두 무효화된다는 뜻은 아니다.)

가설 B — 스크립트 반복 실행

동일한 검증 결과를 full-script cache에서 재사용할 수 없다면, 이후의 블록 검증 또는 재검증 문맥에서 실제 스크립트 실행이 반복될 수 있다.

어떤 사건 사이의 반복인지를 실험에서 특정?

- 메모리 검증 후 블록 연결?
- 블록 disconnect 후 reconnect?
- 경쟁 체인 또는 reorg 처리?
- 동일 블록의 재처리?
- 노드 재시작 후 검증?

Phase 3 : BIP-110 구현 코드 비판적 읽기

가설 C — 실질적인 검증 비용 증폭

캐시 비사용과 스크립트 반복 실행이 실제로 결합되고 공격자가 그 비용을 충분히 크게 구성할 수 있다면,

특정 블록은 정상적인 대조 블록보다 높은 검증 비용을 유발할 수 있다.

이 가설은 아직 다음 사항을 포함하지 않음.

- 공격이 경제적으로 실행 가능한가
- 비용 차이가 실제 운영 장애를 일으킬 만큼 큰가
- 이것을 엄밀한 의미의 poison block이라 부를 수 있는가
- Core와 Knots 사이에서 비용이나 판정이 어떻게 달라지는가

Phase 3 : BIP-110 구현 코드 비판적 읽기

Phase 3에서 확정

grandfathering이 필요한 이유

`flags_per_input`이 도입된 구조

full-script cache 사용 조건의 변화

관련 함수와 조건문의 호출 관계

Todd가 제기한 우려의 코드상 근거

최종발표에서 검증

반복 실행의 실제 발생 여부

반복 실행 횟수

시간·CPU 비용 차이

공격 구성 가능성과 규모

poison-block이라는 명칭의 타당성

Phase 3 : BIP-110 구현 코드 비판적 읽기

[Conclusion]

BIP-110의 `grandfathering`은 과거 UTXO의 소비 가능성을 보존하지만 검증 규칙을 입력별 문맥에 의존하게 만들기 때문에 이 변화는 `flags_per_input`의 도입과 `full-script execution cache` 사용 조건의 변경으로 이어진다.

코드 분석으로 캐시 사용 조건의 변화까지는 확인할 수 있다. 그러나 그것이 실제 스크립트 반복 실행과 유의미한 검증 비용 증폭으로 이어지는지는 정적 분석만으로 확정할 수 없다. 따라서 본 리뷰에서 코드를 중심으로 정적 코드 리뷰를 통해 분석한 로직을 대상으로 이를 A·B·C의 단계적 가설로 분리하고, 동적 계측과 Regtest 벤치마크를 다음 검증 단계로 남긴다.

[Outro : Wrap it up]

Reflection on : “ **Cypherpunk** Is dead ?”

Phase 0 : “BIP-110은 무엇을 제한하려는가?”

Phase 1 : “임의 데이터는 실제로 어디에 들어가는가?”

Phase 2 : “왜 OP_RETURN만 제한해서는 해결되지 않는가?”

Phase 3 : “그렇다면 광범위한 제한 규칙을 구현할 때 어떤 복잡성과 비용이 생기는가?”

[Outro : Wrap it up]

A. “Cypherpunk is dead?” → 사이퍼펑크 정신이 무엇인지? → 현재, 그 정신의 계승자가 누구이며 자유·중립성·책임 가운데 무엇을 우선해야 하는지를 둘러싼 논쟁 속에서 다시 시험받고 있다는 것은 아닌지 ?

B. 인스크립션을 악용된 해킹 사례로 정의함을 근거로 비트코인 개발의 근본 목적인 허가가 필요없고 국경없는 건전한 화폐라는 취지를 지키기 위해 비트코인을 화폐로서 개선하는 일에 우선순위를 다시 집중하고 임의 데이터 지원을 표준화하는 작업을 위해 7가지 제한 규칙 제안 (BIP110)

C. OP_RETURN을 활용한 데이터 삽입 방식은 버전별 정책적 문제와 별개로 표준화된 방식이며 블록 체인에 '임의의 데이터' 삽입이 가능한 공간 5개 영역이 있으며 Witness program 안에서 실행되는 Taproot Script Path를 활용해 데이터를 삽입하는 “인스크립션” 행위에 대한 표준이 존재하지 않고 이를 버그로 규정인정하는 것에 관한 논쟁이 핵심

D. OP_RETURN의 ‘provably unspendable’ + ‘null data’ 정의 정책을 우회하여 witness/tapscript 구조를 이용한 inscription 우회 데이터 삽입 기법이 출현하며 임의 데이터 허용 범위 논쟁을 정책적 문제가 아닌 합의 규칙의 문제로 확장한 RDTs(BIP-110 제안) 등장

E. BIP-110의 grandfathering은 과거 UTXO의 소비 가능성을 보존하지만 검증 규칙을 입력별 문맥에 의존하게 만들기 때문에 이 변화는 flags_per_input의 도입과 full-script execution cache 사용 조건의 변경으로 이어지며 더 나아가 이러한 상황에서 Poison 블록 공격 가능성을 만들어 낼 수 있기 때문에 공격 성립을 입증 혹은 철회하기 위한 가설 설정 및 검증, 실험 설계, 동적 계측 진행 예정.

[감사합니다]

[Q&A]